

Who granted the days, who took them, and who never had them?

An analyst case study — Vantage Wholesale Supply receivables

Reporting date: 2025-12-31 · **Tooling:** T-SQL (SQL Server), Power Query, Excel

Data disclosure. Vantage Wholesale Supply is a fictional industrial parts distributor. Every figure in this study is synthetic, generated by the scripts in `sql/`, and reproducible from them. No confidential data is used and no claim is made about any production system.

1. The situation

Vantage Wholesale Supply sells bearings, fasteners, fluid-power components and MRO consumables on credit to around 400 trade accounts. Receivables run at roughly \$10.8M against credit sales of about \$176,000 a day.

Over the last seven quarters, Days Sales Outstanding has climbed from **55.85 to 63.07 days**. The finance director reads that as collections going slack. The collections manager reads it as Sales having sold longer terms and nobody having re-baselined the target. Both are plausible, both are self-serving, and the argument has been running on assertion because nobody has split the number.

Meanwhile six collectors work a 2,300-line ageing report from the top down by balance.

The question

Of the days DSO has risen, how many did Vantage grant itself through the terms it sold, and how many are customers taking beyond those terms — and given that split, who should each collector call first?

The answer turned out to need a third category that neither side had proposed.

2. Approach

2.1 One definition of what is owed

Everything downstream reads from a single function, so the definition cannot drift into several:

```
OpenBalance = InvoiceAmount - cash applied - discounts taken - credit memos - write-offs
```

The word **applied** is doing real work there. Cash that has reached the bank but has not been matched to an invoice does *not* reduce a balance, because the invoice genuinely is still open — and the date that reduces it is the application date, never the receipt date. Cash banked on 28 December and matched on 4 January does not cure a December ageing bucket, and treating it as if it did is a well-worn way to flatter a month end.

2.2 Two DSOs, on purpose

Method	Reads	Used for
Countback (exhaustion)	63.07 d	The headline. Consumes the balance against the actual months of sales that generated it, so a seasonal dip cannot manufacture a collections crisis.
Classic (balance ÷ daily sales)	61.59 d	The bridge, and any external comparison. It is what a lender or a benchmark quotes.

Quoting a countback figure to someone benchmarking on classic starts an argument you will lose, so both are published and the gap is itself a reported number.

2.3 The bridge, built to be exact

The decomposition is deliberately built on **classic** DSO, because classic decomposes exactly and countback does not. $DSO = TotalAR \div SalesPerDay$, and total AR partitions with no overlap and nothing left over:

Component	Days	Cash	Owner
Granted — not yet due	45.97	\$8,095,793.78	Sales / terms policy
Taken — disputed and past due	0.88	\$154,864.71	Billing / sales
Taken — past due, undisputed	14.74	\$2,596,423.00	Collections
Total (classic DSO)	61.59	\$10,847,081.49	

No residual bar, nothing to plug. The identity is asserted on every test run (UAT-04).

Billing lag is deliberately not in this table. DSO counts from the invoice date, so the days between despatching goods and issuing the invoice happen before the clock starts. Folding them in would break the identity. They *extend* it instead:

CashCycle = BillingLag + DSO_classic = 1.78 + 61.59 = 63.37 days

That is the honest figure — despatch to cash — and it is larger than the number anyone at Vantage is currently reporting.

2.4 Every number computed twice

Each headline figure is implemented independently in T-SQL and again in Excel formulas, then reconciled on the workbook's Validation sheet, in the open. All **27** checks agree. The one worth pausing on: SQL computes countback DSO with a windowed running sum and a TOP 1 ; Excel computes it with a twelve-row helper block, AGGREGATE , MATCH and INDEX . They agree to **0.0011 days**.

3. What the data says

3.1 The split — and the third category

Over the comparable window (2024-06 to 2025-12; the ledger opens in January 2024 and cannot show a representative ageing profile before late April):

	2024-06	2025-12	Change
DSO (countback)	55.85	63.07	+7.22
of which granted (Best Possible DSO)	44.33	46.27	+1.94
of which taken (Average Days Delinquent)	11.52	16.80	+5.28

So the collections manager is more right than the finance director: roughly a quarter of the rise was granted, three quarters is customers paying later.

Except that a large part of the "taken" is not customers at all.

	2024-06	2025-12
Unapplied cash as % of AR	1.79%	3.87%
Unapplied cash	—	\$419,392.13

At \$176,118 of credit sales per day, \$419,392 of unmatched cash is **2.38 days of DSO that no customer is responsible for**. It is Vantage's money, in Vantage's bank account, making Vantage's invoices look unpaid.

The honest decomposition is therefore three ways, not two: of the 16.80 days sitting in "taken", **2.38 are self-inflicted** and 14.42 are genuine customer lateness.

That third category could not even be expressed until cash receipts and cash applications were modelled as separate events. Under the original single-table design, a receipt and its application were the same row, so "money arrived but nobody matched it" had no representation — and the 2.38 days were silently charged to customers.

3.2 The granted quarter was a decision, not an accident

Weighted average terms rose 2.28 days between the second halves of 2024 and 2025. A shift-share decomposition separates *who bought* from *what they were sold*:

Effect	Days	Reading
Mix — same terms, different customers buying	−0.33	Slightly offsetting
Rate — same customers, longer terms	+1.71	A policy change
Interaction	+0.90	Re-papered customers also grew their share
Total ΔWAT	+2.28	

All 400 customers traded in both periods, so this is not composition. The same accounts were moved onto longer terms, and the accounts that were moved bought more. That is a commercial decision someone made; it should be priced and capped rather than discovered in the DSO line.

3.3 The DSO target cannot be hit — arithmetically

	Days
DSO target	45.00

Days	
Weighted average terms actually sold	46.68
Collection allowance the target leaves	-1.68

If every customer paid **exactly on the due date**, DSO would settle near 46.7 and the 45-day target would still be missed. The target sits below the floor the terms mix permits.

A target nobody can reach does not motivate a collections team. It teaches them the scorecard is theatre, which costs more than the target was ever worth.

3.4 Three metrics are Green or Amber while a cohort inside them has failed

This is the pattern worth taking away from the whole engagement.

Metric	Portfolio	Status	The cohort underneath
Promise kept rate	86.60%	Green	High-risk tier: 54.57% by value, 40.79% by count
Billing lag	1.78 d	Green	Southeast DC: 3.22 d , drifting 1.56 → 4.78 through 2025
Credit utilisation	17.70%	Green	One account over limit at 106% ; three above the 80% target

Every one of these aggregates is arithmetically correct and every one is operationally useless on its own. A portfolio average cannot breach until the whole book does; by then the cohort that caused it has been failing for a year. Each of these three now ships with its cohort cut attached to the same report, because a number that can only be read one way is a number that will be read the wrong way.

The promise figure is the sharpest: **255 broken promises worth \$660,504**, 239 of them in one risk tier, sitting underneath a portfolio figure that clears its target.

3.5 Collections effectiveness is flattered by write-offs

The textbook Collection Effectiveness Index measures whether AR went down — and AR goes down identically whether a customer wired the money or a controller wrote the balance off. That makes the standard metric improvable by giving up.

	2024-06	2025-12
CEI (book)	71.80%	66.91%
CEI (cash only)	71.05%	65.26%

	2024-06	2025-12
Paper collections gap	0.75 pts	1.65 pts

Write-offs were zero until April 2025 and total **\$702,389** since. Both figures are below the 75% warning threshold; the cash figure is further below, and it is the honest one.

3.6 The receivables balance is mis-stated by \$371,040

The data-quality gate **fails**, and deliberately is not suppressed:

Defect	Count	Value
Duplicate cash receipts	96	\$371,039.99
Over-applied invoices (their effect)	101	\$371,039.99
Unapplied cash older than 90 days	65	\$265,487.11
Receipts dated before their invoice	27	\$100,496.95
Invoices issued before despatch	20	\$76,841.74
Due date inconsistent with terms	52	\$12,727.40

The duplicates are the material one: **3.421% of open AR**, against a 1.000% tolerance. Note that `DUPLICATE_RECEIPT` and `OVER_APPLIED_CASH` report the *same* \$371,039.99 — one is the cause and the other its effect on the ledger. Only the effect counts toward exposure, or the same money would be charged twice.

4. The operational control

A ranked list of every account with a past-due balance is a ledger with a sort order. 272 accounts is two thirds of the customer base, and six collectors cannot call that in a day.

The queue ranks on **collectable exposure** — ageing-weighted past-due dollars, less 75% of the ageing-weighted *disputed* dollars, less cash already banked against the account — and then bounds itself to what the team can work:

Action	Accounts	Past due	Collectable exposure	On today's list
<code>ESCALATE</code>	19	\$758,532	\$359,632	19

Action	Accounts	Past due	Collectable exposure	On today's list
COLLECTION_CALL	26	\$476,443	\$155,712	15
STANDARD_DUNNING	100	\$634,679	\$144,147	21
FINAL_NOTICE	19	\$356,211	\$80,901	5
RESOLVE_DISPUTE	45	\$195,694	\$36,407	5
COURTESY_REMINDER	12	\$26,036	\$6,509	0
APPLY_CASH	51	\$303,692	\$5,011	0
Total	272	\$2,751,288	\$788,318	65

65 accounts — 10 to 13 per collector — carrying 81.3% of the collectable exposure.

Four design decisions are worth naming:

- **APPLY_CASH takes precedence over every other action.** 51 accounts have cash already banked against them covering at least half their past-due balance. Ringing a customer who has already paid is the single most damaging call a collections team can make, and it is not a collections job at all — it is a cash application job. These accounts are routed out of the call list entirely: they never take one of a collector's daily slots, and **UAT-20 asserts at every month-end that none of them reaches a collector's workload.**
- **Disputes are discounted, not ranked.** Chasing cash on a disputed invoice does not work; that balance needs the dispute resolving, which is a different team's job.
- **Two broken promises in 90 days escalates,** regardless of balance. It is the earliest reliable distress signal in the data and it costs nothing to collect.
- **Credit hold is a flag, not an action.** Holding orders sits alongside a collections action rather than replacing it. As a competing branch it was also unreachable — every over-limit account already qualified for escalation — so the control silently did nothing.

The second workload

Collections gets a call list. Cash application gets its own, ranked oldest-first:

97 accounts hold \$419,392 of unmatched cash. 57 receipts have no application against them at all. The oldest of those has been sitting for 668 days; the oldest receipt with any unapplied remainder at all goes back 679. \$409,640 of past-due balance would be wrongly chased if nobody applies it first.

This is the cheapest money in the entire analysis: no negotiation, no concession, no relationship cost. It is already in the bank.

5. Recommendations

#	Recommendation	Owner	Basis	Value
1	Apply the cash before making a single call	Cash application	97 accounts hold \$419,392 unmatched; 57 receipts wholly unapplied; oldest wholly-unapplied 668 days; oldest with any remainder 679	2.38 days of DSO and \$409,640 of wrongly-chaseable balance, at no customer cost
2	Fix duplicate receipt posting	Controller	96 duplicate bankings mis-state AR by 3.421%	\$371,040 of AR correctly stated
3	Reset the DSO target to terms + allowance, about 55 days	CFO	Target 45 sits 1.68 days below the 46.68-day terms floor	Restores a scorecard the team can act on
4	Work the bounded queue , starting with the 19 escalations	Collections manager	65 accounts carry 81.3% of collectable exposure	Focuses 6 collectors on \$640,540 of the \$788,318 at stake
5	Report promise kept-rate by risk tier, never as a portfolio figure	Collections manager	86.60% overall conceals 54.57% in the High tier: 239 of the 255 broken promises, worth \$607,759	Turns a Green light into an early-warning system
6	Investigate Southeast invoice posting	Operations	3.22 days against a 1.50 baseline elsewhere, drifting all year	\$50,477 of cash, recoverable by changing a batch schedule
7	Price and cap the extended-terms programme at the Q1 review	Sales director	Shift-share isolates a deliberate +1.71-day rate effect across the same 400 customers	Makes ~\$301k of working capital a priced decision rather than a drift
8	Measure collections on CEI (cash), not CEI (book)	CFO	\$702,389 of write-offs currently score as collections	Removes an incentive to improve the metric by giving up

Recommendations 1 and 2 are the same root cause — cash application is the broken function — and together they account for **\$790,432** of mis-stated or unusable receivables. Neither needs a customer conversation.

6. What went wrong during the build, and what it cost

6.1 A false finding that survived every balance check

The generator paid 60% of a partial invoice on the first instalment and emitted the second only when the invoice was *not* a discount-taker — the amount rule and the row filter disagreed about the same condition. 208 invoices kept a 40% residual that could never clear.

Because `2/10NET30` is the only terms code carrying a discount, the whole defect landed on one cohort and presented as a clean commercial insight:

"47.4% of open discount-terms invoices are more than 90 days past due, against 2.3% on NET30."

Every row was internally consistent. The settlement identity held to the cent. The ageing buckets summed correctly. No balance-level check could see it. It surfaced only when open AR was cut by **ageing and terms together** and the result was too extreme to be plausible behaviour.

The lesson: a defect that is internally consistent will pass every internal check. Cross-cuts that no single check covers are where they surface — and an implausible result deserves investigation before it deserves a slide.

6.2 A dashboard that reported success while broken

The Excel build script completed without error and printed "Saved". Opening the file showed `#REF!` through the entire Priority Queue: Excel does not defer a reference to a sheet that does not exist yet — it rewrites the reference and moves on, and the Calc sheet had been written before the sheets it referenced were created.

A second fault was worse because it was quieter: the ageing matrix's dimension lists were written over the countback block's "covers the balance?" flags, so `MATCH` skipped the overwritten rows and returned Best Possible DSO of **42.75** instead of **46.19**, the correct figure on the data of the time (46.27 on the final dataset) — a plausible number, not an error.

Both were found by a separate script that opens the saved workbook, forces a recalculation and reads the values back. **A build that finishes without throwing has proved it ran, not that it produced a working file.**

6.3 A published figure that disagreed with its own components

`AvgDaysDelinquent` is `DSO - BPDSO`. All three were rounded to two decimals independently, so the page published 63.07, 46.27 and 16.81 — and a reader doing the subtraction got 16.80. Caught by UAT-11.

The fix is a rule now applied in both implementations: **a derived figure is computed from the components as published, not from their unrounded intermediates**. The same class of defect had already appeared once, where `fn_DSO` divided before multiplying while `fn_DS0Bridge` multiplied before dividing, and the two published a classic DSO that differed by a cent.

6.4 A test whose premise my own feature invalidated

UAT-02 asserted as-of integrity by subtracting all cash applied after 30 June from the December figure. Once applications could legitimately fall *after* the reporting date — the December-banked, January-applied case the new model exists to represent — that subtraction removed \$165,415.55 that had never been in the December total. **The test failed; the data was right.**

It was replaced with a direct recomputation rather than a subtraction. Worth recording because the instinct on a red test is to suspect the code, and here the code was the only correct part.

7. What I would do differently

- **Model cash receipt and cash application separately from the start.** This was originally deferred as a limitation, and closing it changed a headline conclusion: 2.38 days of DSO moved out of "customers are paying late" and into "we have not applied our own cash". A modelling shortcut that cannot represent a category will silently attribute that category to something else.
 - **Cut the data by pairs of dimensions during profiling.** The instalment defect was invisible to single-dimension checks and obvious the moment ageing met terms. Every cohort finding in \$3.4 came from the same habit, applied deliberately rather than accidentally.
 - **Write the workbook validator before the workbook.** It found two defects in the first file it opened. Had it existed first, the build would never have reported a success it had not earned.
 - **Treat rounding as a published contract, not a formatting choice.** Two of the defects above were rounding inconsistencies that produced figures which did not add up on the page.
-

8. How to verify any figure in this study

```
sqlcmd -S <server> -E -d master -i sql/01_create_schema.sql
sqlcmd -S <server> -E -d master -i sql/02_generate_synthetic_data.sql
sqlcmd -S <server> -E -d VantageAR -i sql/03_core_views.sql
sqlcmd -S <server> -E -d VantageAR -i sql/04_data_quality_checks.sql
sqlcmd -S <server> -E -d VantageAR -i sql/05_kpi_views.sql
sqlcmd -S <server> -E -d VantageAR -i sql/06_dso_bridge.sql
sqlcmd -S <server> -E -d VantageAR -i sql/07_stored_procedures.sql
sqlcmd -S <server> -E -d VantageAR -i sql/08_uat_test_cases.sql # 25 cases

EXEC dbo.usp_ARScorecard @AsOf = '2025-12-31';
EXEC dbo.usp_DSABridgeReport @AsOf = '2025-12-31';
EXEC dbo.usp_PriorityQueue @AsOf = '2025-12-31', @TodaysWorklistOnly = 1;
EXEC dbo.usp_CashApplicationWorklist @AsOf = '2025-12-31';
EXEC dbo.usp_PromiseReport @AsOf = '2025-12-31', @GroupBy = 'RiskTier';
EXEC dbo.usp_BillingLagReport @AsOf = '2025-12-31';
EXEC dbo.usp_RunDataQualityChecks;

powershell erp_extracts/Export_Extracts.ps1
powershell excel/Build_Workbook.ps1
powershell excel/Validate_Workbook.ps1 # 27 checks
```

The generator is deterministic — hash-keyed rather than `RAND()` — so a rebuild reproduces the dataset byte for byte, and UAT-16 asserts the fingerprints.

Appendix — headline figures at 2025-12-31

Metric	Value	Target	Warning	Status
DSO (countback)	63.07 d	45.00	55.00	Red
Average days delinquent	16.80 d	8.00	15.00	Red
Collection effectiveness (book)	66.91%	85.00	75.00	Red
Collection effectiveness (cash)	65.26%	—	—	—
Past due share of AR	25.36%	15.00	25.00	Red
90+ share of AR	3.35%	3.00	6.00	Amber
Unapplied cash % of AR	3.87%	0.50	1.50	Red
Billing lag (days)	1.78 d	2.00	4.00	Green*

Metric	Value	Target	Warning	Status
Promise kept rate	86.60%	80.00	65.00	Green*
Credit utilisation (portfolio)	17.70%	80.00	100.00	Green*
Mean open dispute age	28.13 d	15.00	30.00	Amber

* Green at portfolio level only. Southeast billing lag is 3.22 days; the High-risk promise kept rate is 54.57%; one account (C0297) is over its credit limit at 106.10%; three are above the 80% target, one at 106%. See §3.4.

Ageing bucket	Invoices	Balance	Share
Current	1,672	\$8,095,793.78	74.6%
1–30 days	414	\$1,839,519.41	17.0%
31–60 days	101	\$405,938.13	3.7%
61–90 days	34	\$142,966.39	1.3%
90+ days	125	\$362,863.78	3.3%
Total	2,346	\$10,847,081.49	100%

Cash cycle	Days
Billing lag (despatch → invoice)	1.78
Classic DSO (invoice → cash applied)	61.59
True cash cycle (despatch → cash)	63.37
of which unapplied cash Vantage already holds	2.38